CSE 4513

Lec – 12

What is SW Testing

DEFINITION

Testing is the process of executing a program with the intent of finding errors.

- Myers

Testing is a support function that helps developers look good by finding their mistakes before anyone else does.

– *James Bach*

Software testing is a process that detects important bugs with the objective of having better quality software.

SW TESTING

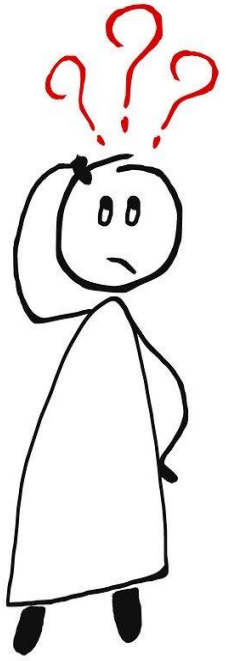
- ✓ Software Testing is a process where the software is evaluated to determine that it meets the **user needs (validation)** and that the **process to build the software was followed correctly(verification).**
- ✓ in other way,
- Validation: Are we building the right product? (Does the software meet the user's needs and work correctly?)
- Verification: Are we building the product right? (Is the software built according to the design?)

TESTING : WHY DO WE PERFORM

Testing fulfills two primary purposes:

- ✓ to demonstrate quality or proper behavior;
- ✓ to detect and fix problems.

finding and eventually fixing defects is the Number one reason for software testing. But the truth is, no Software is free from Defect.



If all software is released to customers with defects, why should we spend so much time, effort, and money on testing?

TESTING: WHY IS IT IMPORTANT

- ✓ Most of us have had an experience with systems that don't work as expected. This can have a huge impact on a company, including:

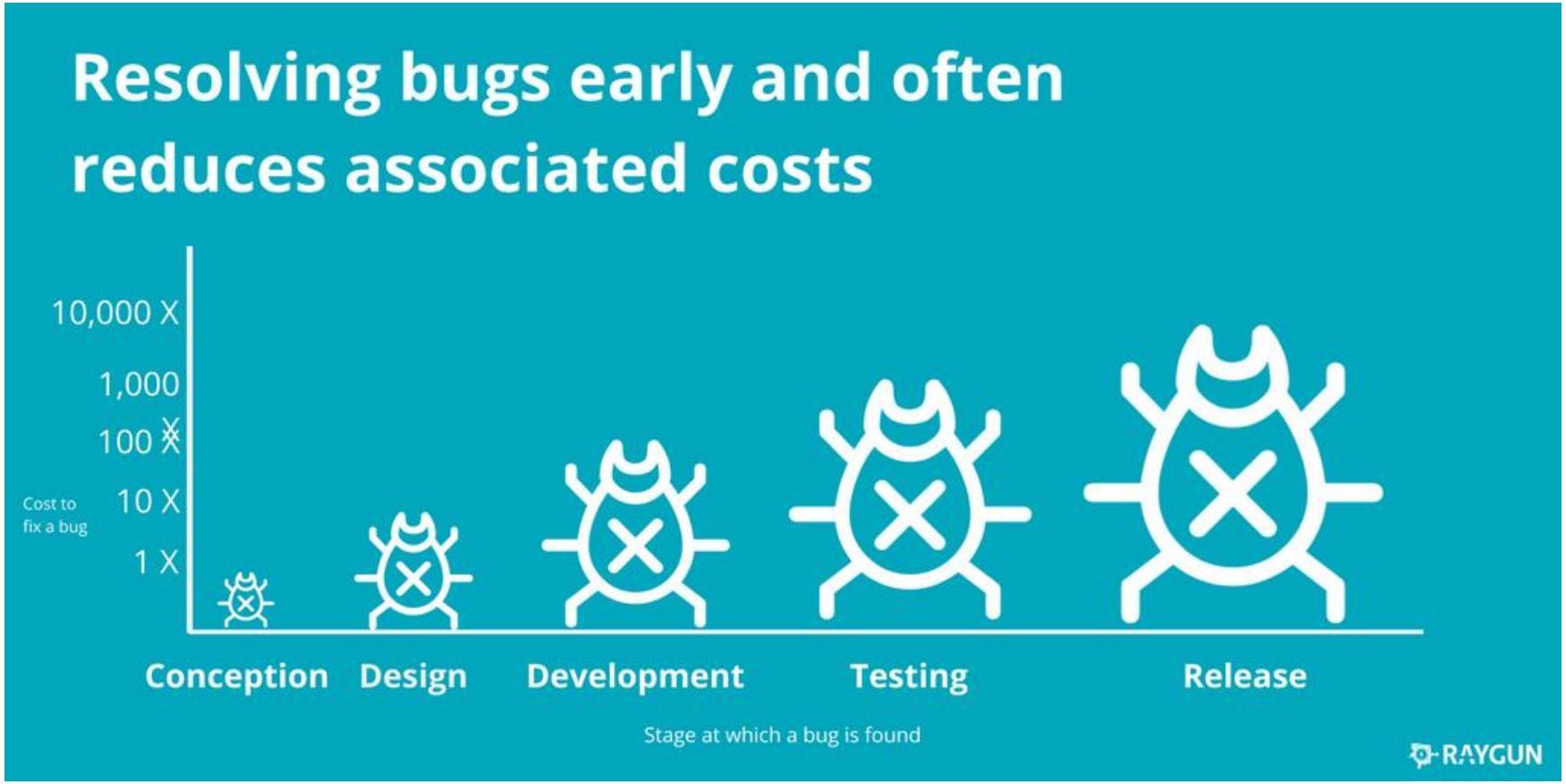
  Financial loss – losing business is bad enough but in extreme cases there are financial penalties for non-compliance to regulations

  Increased costs – these tend to be the result of having to implement a manual workaround but can include staff not being able to work due to a fault or failure and the cost of fixing the issues after go-live

  Reputational loss – if the company is unable to provide service to their customers due to software problems then the customers will probably take their business elsewhere

It is important to test the system to ensure it is error free

Cost of Late Testing



Early testing can save significant money for the company!

SOFTWARE TESTING — MYTHS AND FACTS

Myth Testing is a single phase in **SDLC** .

Truth in reality, testing starts as soon as we get the requirement specifications for the software. And the testing work continues throughout the **SDLC**, even post-implementation of the software.

Myth Testing is easy..

Truth is not easy, as they have to plan and develop the test cases manually and it requires a thorough understanding of the project being developed with its overall design. Overall, testers have to shoulder a lot of responsibility which sometimes make their job even harder than that of a developer..

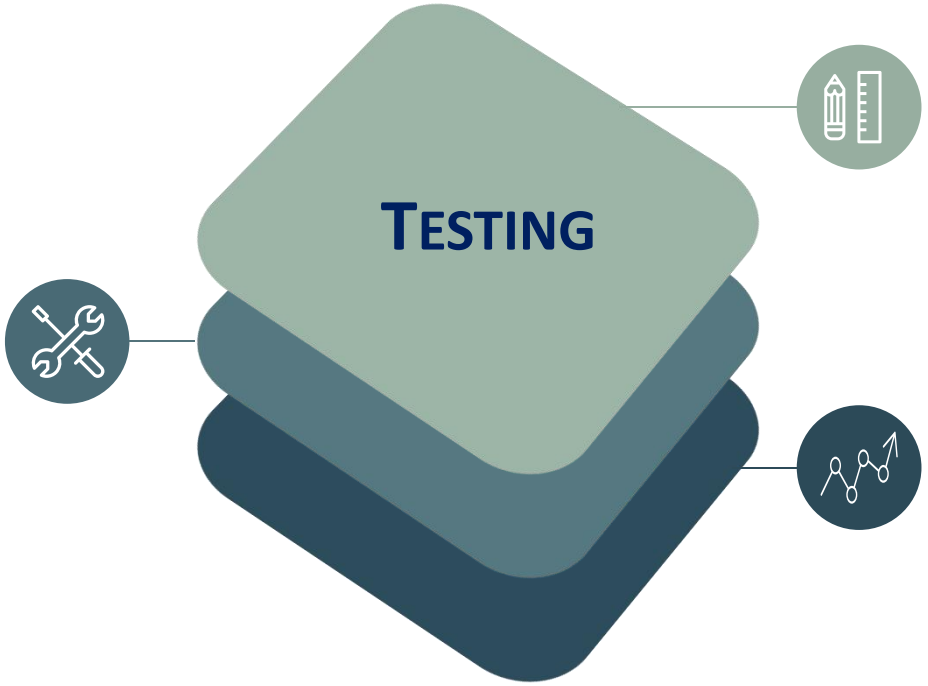
SOFTWARE TESTING — MYTHS AND FACTS

Myth	Software development is worth more than testing.
Truth	Testing is a complete process like development, so the testing team enjoys equal status and importance as the development team.
Myth	Complete testing is possible.
Truth	there are many things which cannot be tested completely, as it may take years to do so.
Myth	Testing starts after program development.
Truth	the work of a tester begins as soon as we get the specifications. The tester performs testing at the end of every phase of SDLC in the form of verification and plans for the validation testing.

SOFTWARE TESTING — MYTHS AND FACTS

Myth	The purpose of testing is to check the functionality of the software.
Truth	quality does not always imply checking only the functionalities of all the modules. There are various things related to quality of the software, for which test cases must be executed..
Myth	Anyone can be a tester.
Truth	??????????????????

GOALS OF SOFTWARE TESTING



Immediate Goals
- Bug discovery
- Bug prevention

Post-implementation Goals
- Reduced maintenance cost
- Improved testing process

Long-term Goals
- Reliability
- Risk management
- Quality
- Customer satisfaction

TESTING TERMINOLOGY

Failure

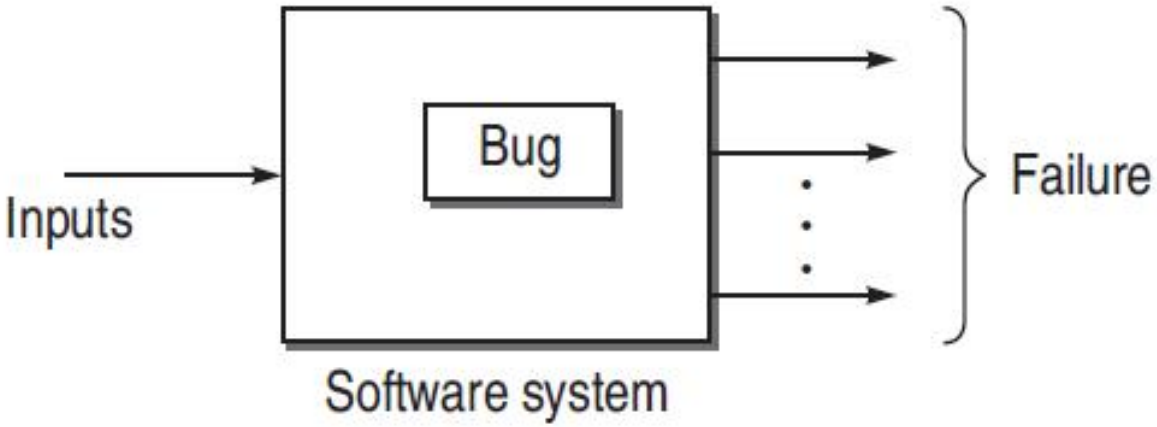
Is the inability of a system or component to perform a required function according to its specification.

Fault/Defect/Bug

is an **error** in code due to incorrect design, logic, or implementation where the software does not behave as expected.

Error

Whenever a development team member makes a mistake in any phase of SDLC, errors are produced. It might be a typographical error, a misleading of a specification, a misunderstanding of what a subroutine does, and so on.



Why Do Bugs Occur?

Faulty definition of requirements

- Erroneous requirement definitions
- Absence of important requirements
- Incomplete requirements
- Unnecessary requirements included

Client-developer communication failures

- Misunderstanding of client requirements presented in writing, orally, etc.
- Misunderstanding of client responses to design problems

Coding errors

- Errors in interpreting the design document, errors related to incorrect use of programming language constructs, etc.

Why Do Bugs Occur?

Deliberate deviations from software requirements

- Reuse of existing software components from previous projects without complete analysis
- Functionality omitted due to budget or time constraints
- "Improvements" to software that are not in requirements

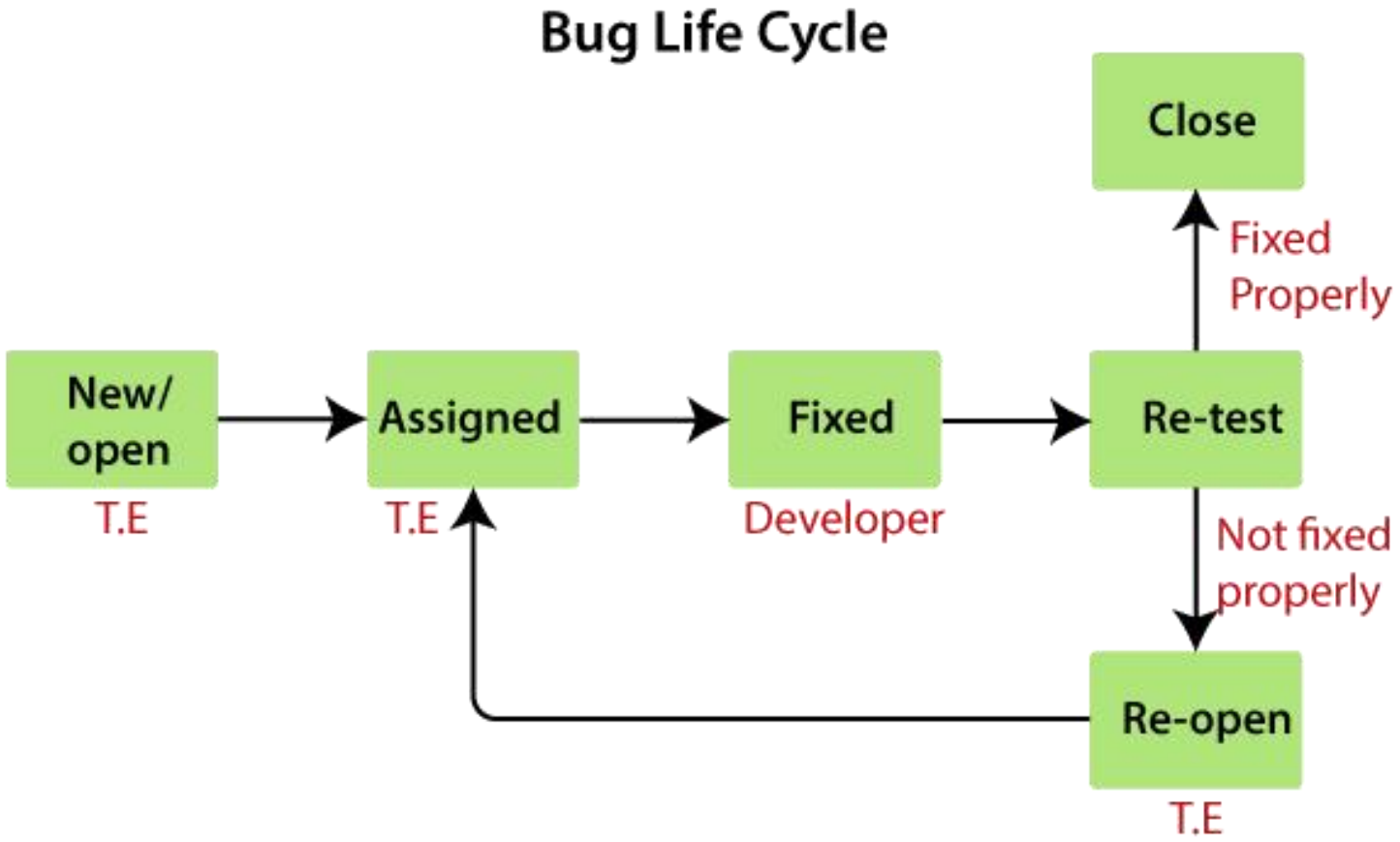
Logical design errors

- Errors in interpreting the requirements into a design (e.g. errors in definitions of boundary conditions, algorithms, reactions to illegal operations,...)

Shortcomings of the testing process

- Incomplete test plan
- Failure to report all errors/faults resulting from testing
- Incorrect reporting of errors/faults
- Incomplete correction of detected errors

DEFECT/BUG LIFE CYCLE

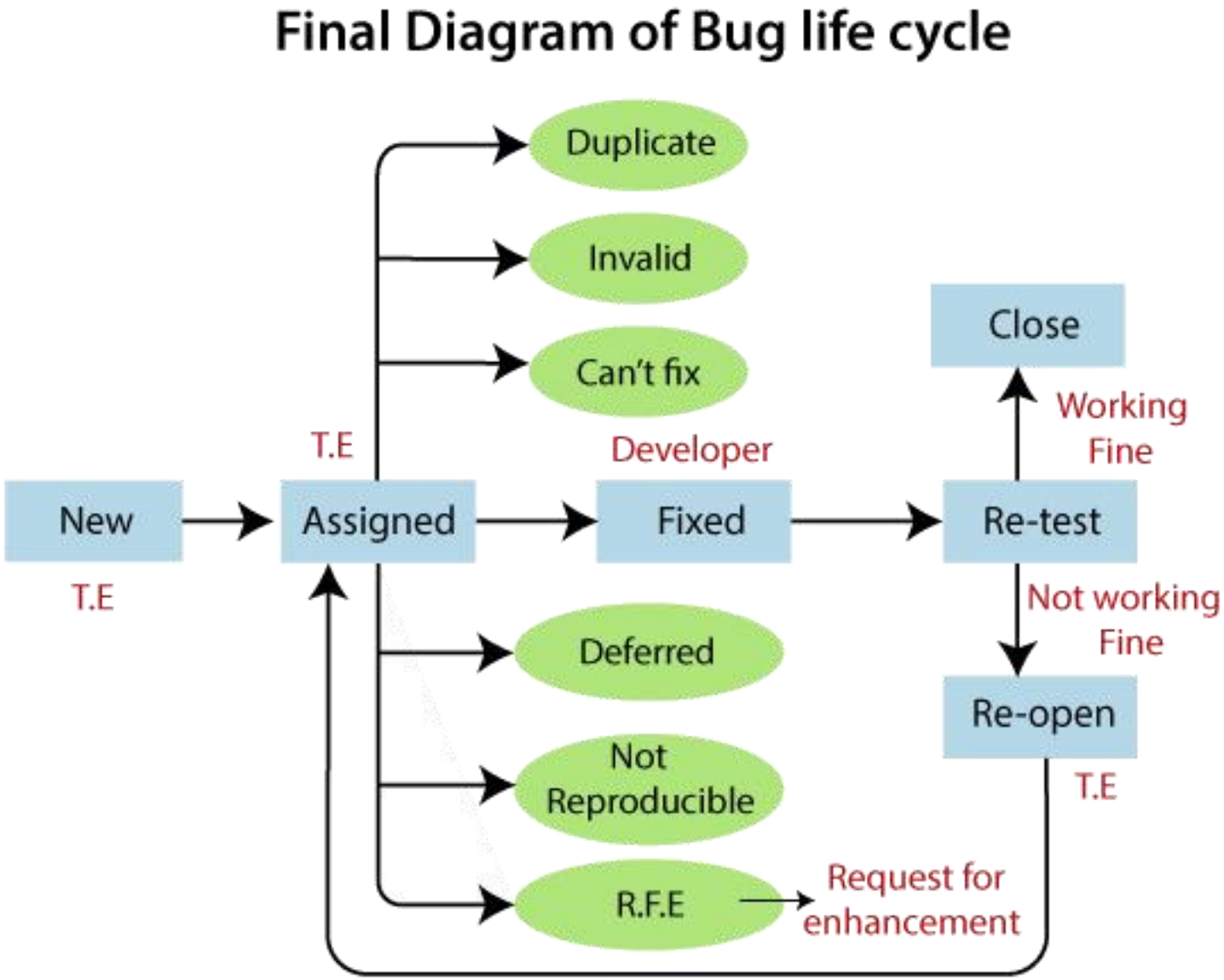


Others status of the bug

Following are the different status of the bug:

- ✓ Invalid/rejected
- ✓ Duplicate
- ✓ Postpone/deferred
- ✓ Can't fix
- ✓ Not reproducible
- ✓ RFE (Request for Enhancement)

DEFECT/BUG LIFE CYCLE

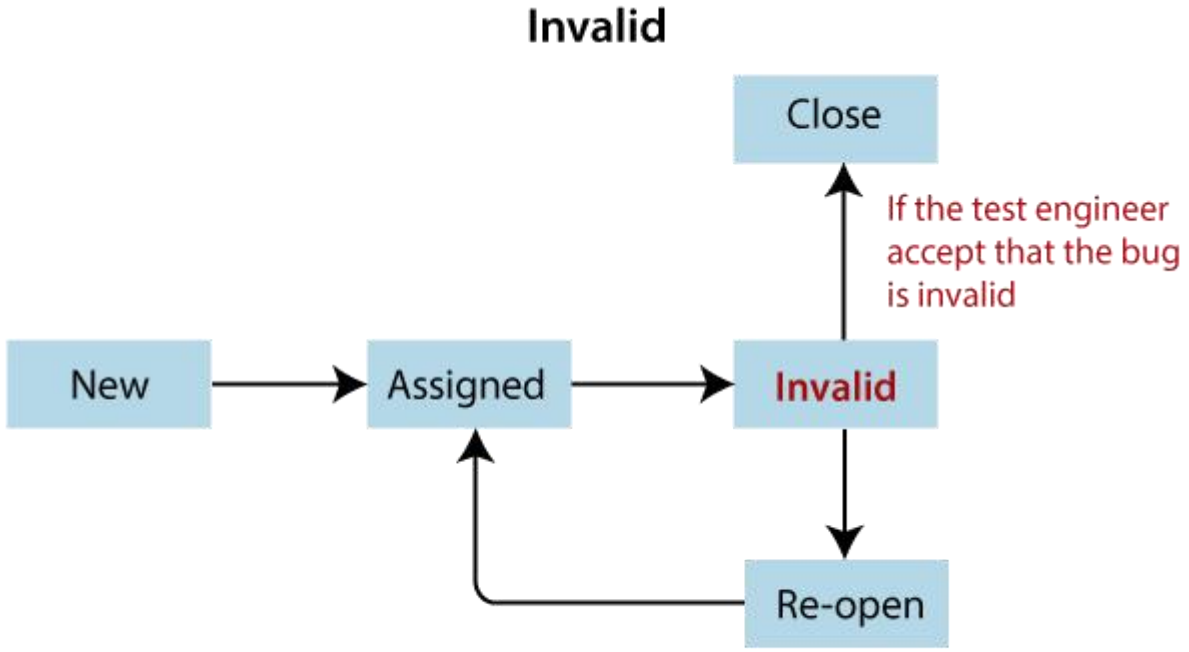


Invalid Bug

Any bug which is not accepted by the developer is known as an invalid bug.

Reasons for an invalid status of the bug :

1. Test Engineer misunderstood the requirements
2. Developer misunderstood the requirements



Research Area:

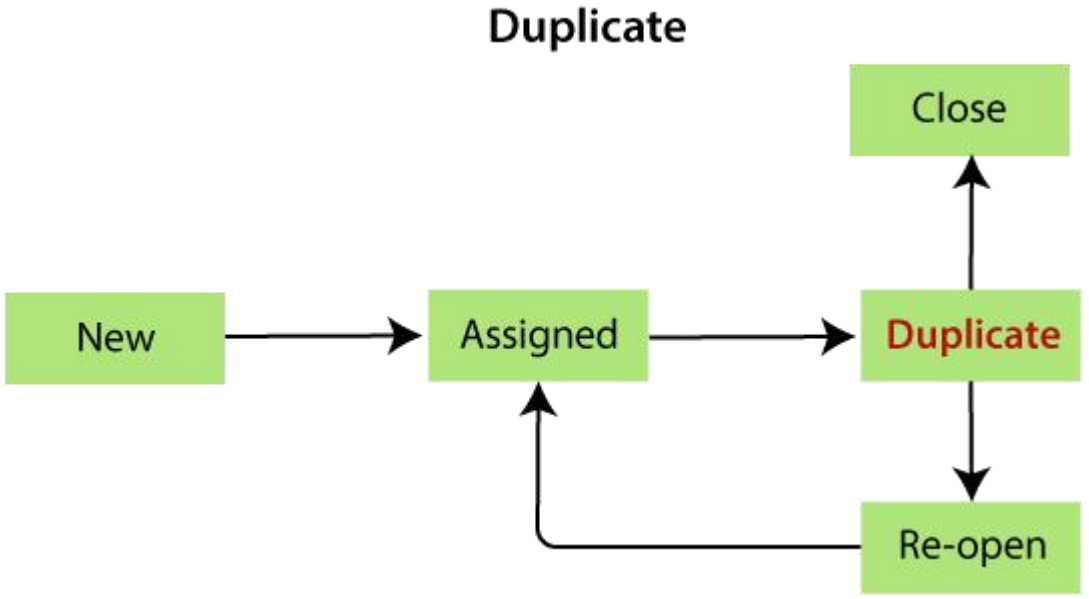
- A data-driven approach for understanding invalid bug reports: An industrial case study
- Deep Learning-Based Valid Bug Reports Determination and Explanation

DUPLICATE BUG

When the same bug has been reported multiple times by the different test engineers are known as a duplicate bug.

Reasons for an duplicate status of the bug :

1. Common features
2. Dependent Modules



Research Area:

- Characterizing Duplicate Bugs: An Empirical Analysis
- Duplicate Bug Report Detection and Classification System Based on Deep Learning Technique
- Duplicate Bug Report Detection Using Dual-Channel Convolutional Neural Networks

DUPLICATE BUG

To avoid the duplicate bug:

If the Developer got a bug, then he/she will go to the bug repository and search for the bug and also check whether the bug exist or not.

If the same bug exist, then no need to log the same bug in the report again.

Or

If the bug does not exist, then log a bug and store in the bug repository, and send to Developers and Test Engineers adding them in [CC].

NOT REPRODUCIBLE

These are the bug where the developer is not able to find it, after going through the navigation step given by the test engineer in the bug report.

Reasons for the not reproducible status of the bug

1. **Incomplete bug report**
   The Test engineer did not mention the complete navigation steps in the report.
2. **Environment mismatch**
   Environment mismatch can be described in two ways:
   - Server mismatch
   - Platform mismatch

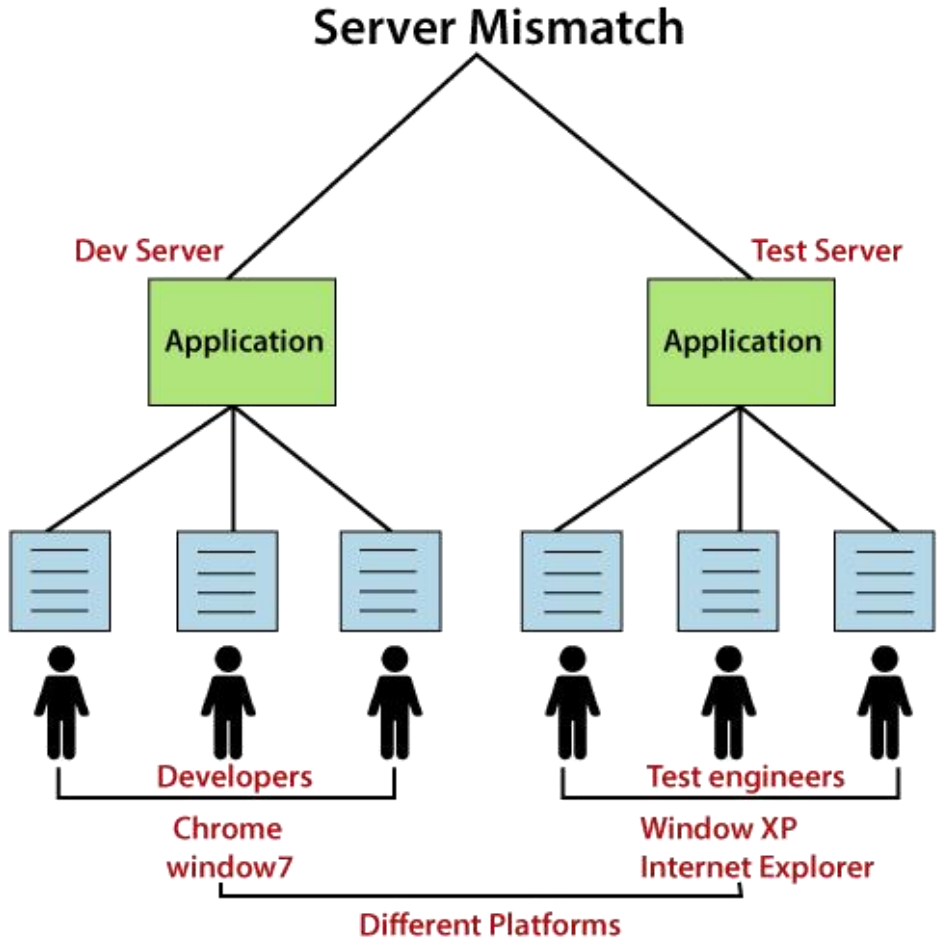
NOT REPRODUCIBLE

Server mismatch

Test Engineer is using a different server (**Test Server**), and the Developer is using the different server (**Development Server**) for reproducing the bug

Platform mismatch:

Test engineer using the different Platform (**window 7 and Google Chrome browser**), and the Developer using the different Platform (**window XP and internet explorer**) as well.



NOT REPRODUCIBLE

Data mismatch

Different Values used by test engineer while testing & Developer uses different values.

For example:

The requirement is given for admin and user.

Test engineer(user) using the below requirements:	**the Developer (admin)** using the below requirements:
User name → abc Password → 123	User name → aaa Password → 111

NOT REPRODUCIBLE

Build mismatch

The test engineer will find the bug in one Build, and the Developer is reproducing the same bug in another build. The bug may be automatically fixed while fixing another bug.

Inconsistent bug

The Bug is found at some time, and sometime it won't happen.

Solution for inconsistent bug:

As soon as we find the bug, first, **take the screenshot**, then developer will **re-confirm** the bug and fix it if exists.

NOT REPRODUCIBLE

Research Area:

- Works for Me! Cannot Reproduce–A Large Scale Empirical Study of Non-reproducible Bugs

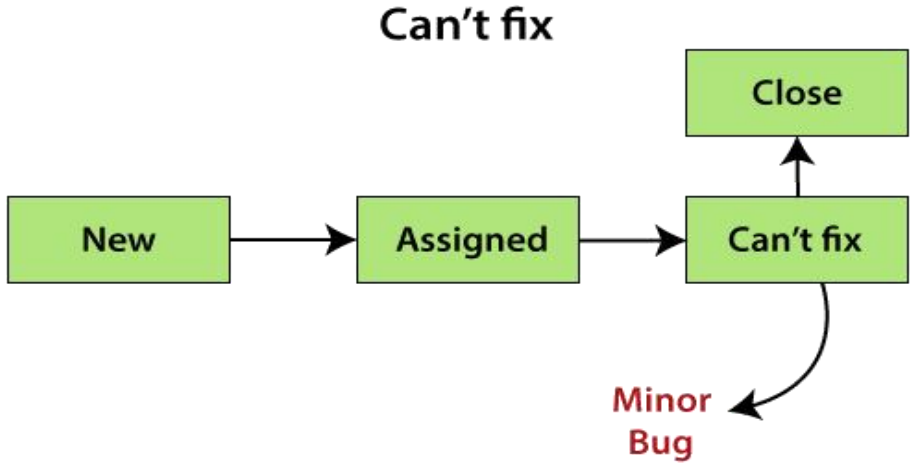
- Why are Some Bugs Non-Reproducible?:–An Empirical Investigation using Data Fusion–

CAN'T FIX

When Developer accepting the bug and also able to reproduce, but can't do the necessary code changes due to some constraints.

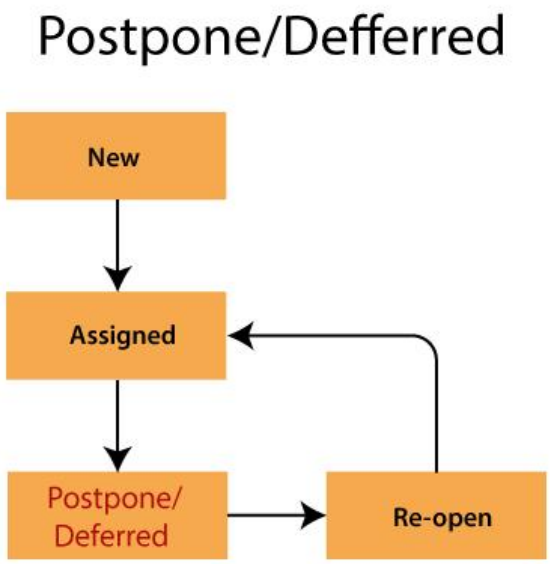
Reasons for the can't fix status of the bug

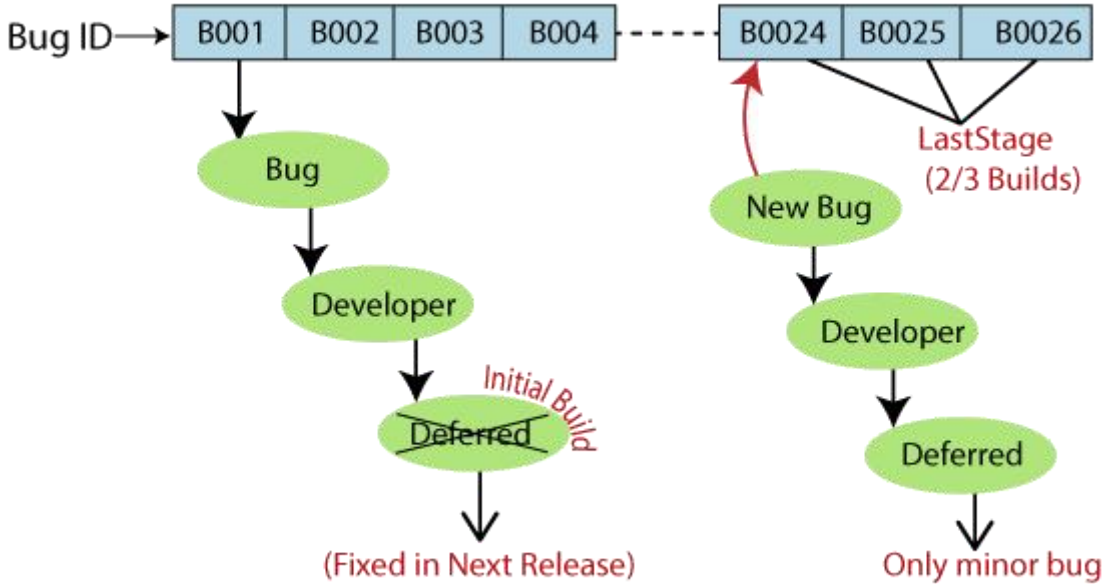
1. No technology support
2. The Bug is in the core of code (framework)
3. The cost of fixing a bug is more than keeping it.



DEFERRED/POSTPONED

The deferred/postpone is a status in which the bugs are postponed to the future release due to time constraints.



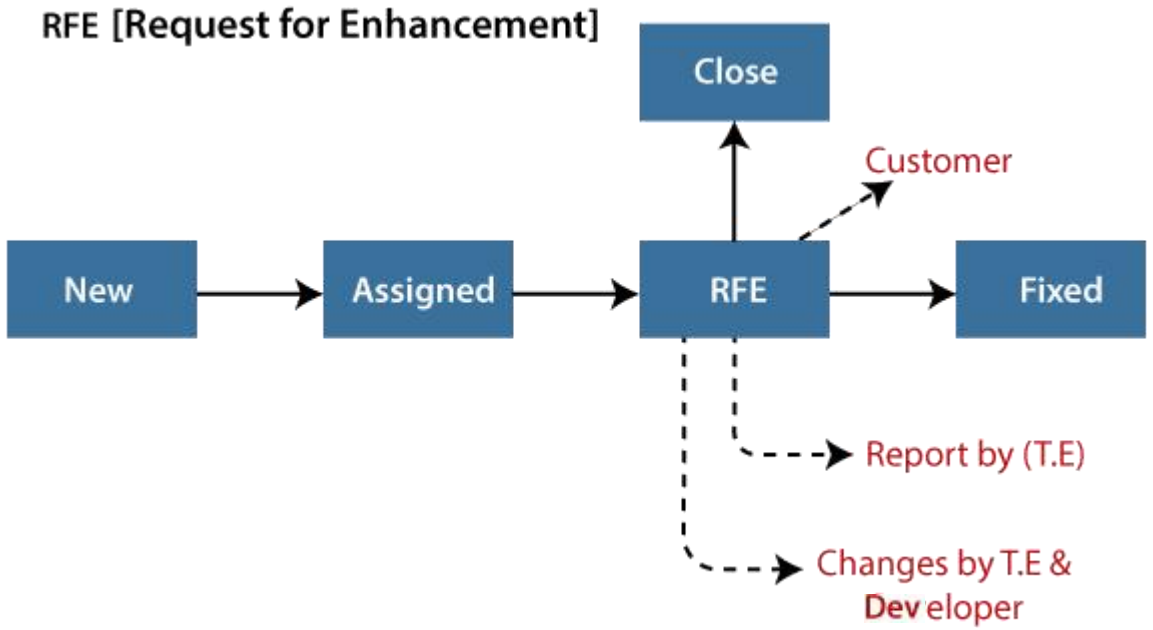


The **Bug ID-B001** bug is found at the initial build, but it will not be fixed in the same build, it will postpone, and **fixed in the next release**.

And **Bug ID- B0024, B0025, and B0026** are those bugs, which are found in the last stage of the build, **and they will not be fixed because these bugs are the minor bugs**.

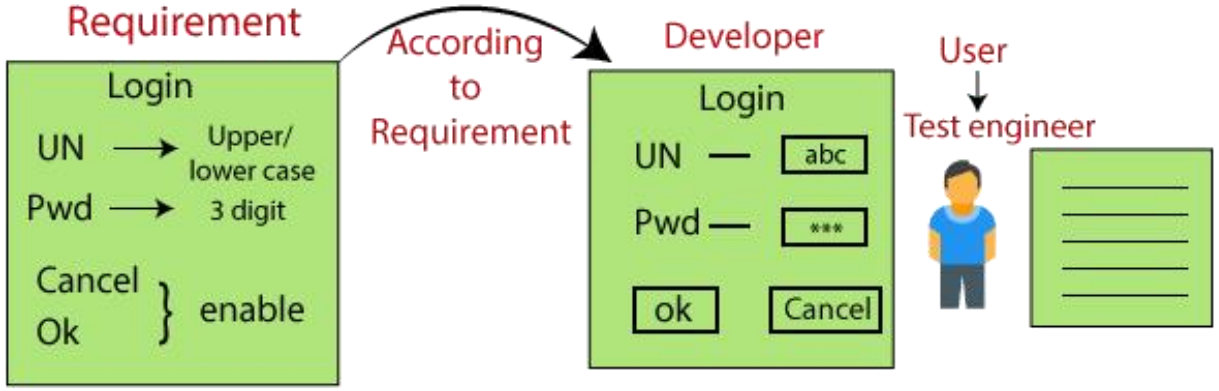
RFE (REQUEST FOR ENHANCEMENT)

These are the suggestions given by the test engineer towards the enhancement of the application in the form of a bug report. The RFE stands for Request for Enhancement.



RFE (Request for Enhancement)

As we can see in the below image that the test engineer thinks that the look and feel of the application or software are not good because the test engineer is testing the application as an end-user, and he/she will change the status as RFE.



And if the customer says Yes, then the status should be Fix.

Or

If the customer says no, then the status should be Close.

BUG SEVERITY

The impact of the bug on the application is known as severity.

It can be a blocker, critical, major, and minor for the bug.

Blocker: if the severity of a bug is a blocker, which means we cannot proceed to the next module, and unnecessarily test engineer sits idle.

Critical: if it is critical, that means the main functionality is not working, and the test engineer cannot continue testing.

Major: if it is major, which means that the supporting components and modules are not working fine, but test engineer can continue the testing.

Minor: if the severity of a bug is major, which means that all the U.I problems are not working fine, but testing can be processed without interruption.

BUG PRIORITY

- ✓ Priority is important for fixing the bug or which bug to be fixed first or how soon the bug should be fixed.
- ✓ It can be urgent, high, medium, and low.

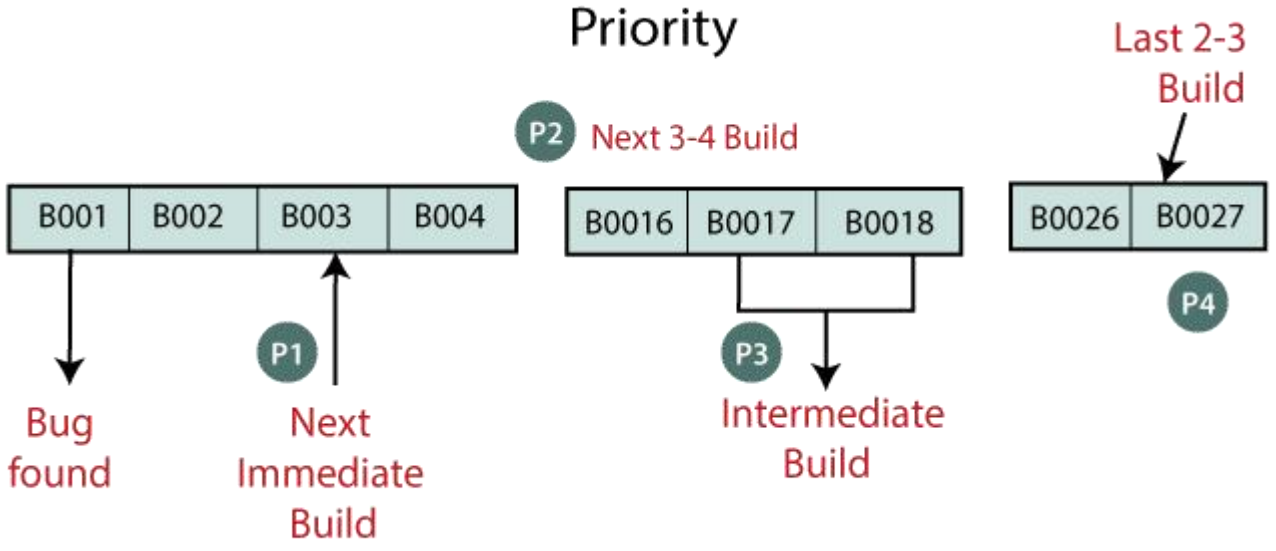
High: it is a major impact on the customer application, and it has to be fixed first.

Medium: In this, the problem should be fixed before the release of the current version in development.

Low: The flow should be fixed if there is time, but it can be deferred with the next release.

EXAMPLE OF SEVERITY AND PRIORITY

- ✓ When the bug is just found, it will be fixed in the next immediate build, and give the Priority as P1 or urgent.
- ✓ If the Priority of the bug is P2 or high, it will be fixed in the next 3-4 builds.
- ✓ When the Priority of the bug is P3/medium, then it will be fixed in the intermediate build of the application.
- ✓ And at last, if the Priority is P4/low, it will be fixed in the last 2-3 build of the software as we can see in the below image:



EXAMPLE OF SEVERITY AND PRIORITY

If we take the case of a email module, then the severity and Priority could depend on the application like as we can see in the below image:

Blocker

Severity	Requirement	Priority
Critical	Login	[P1]
Critical	Compose	[P1]
Critical	Inbox	[P1]
Major	Send Item	[P2]
Major	Trash	[P3]
Minor	Help	[P3]
Minor	Logout	[P4]

WRITING DEFECT/BUG

What makes a good defect report?

Often defect reports do not have enough information, or they are inaccurate and so they are either rejected or when reviewed later, it is impossible to say what the actual problem was.

Parameters of a Defect/Bug

The Following details should be part of a Bug:

- ✓ Date of issue, author, approvals and status.
- ✓ Severity and priority of the incident.
- ✓ The associated test case that revealed the problem
- ✓ Expected and actual results.
- ✓ Description of the incident with steps to Reproduce
- ✓ Status of the incident
- ✓ Conclusions, recommendations and approvals.

CHARACTERISTICS OF A GOOD BUG REPORT

- ✓ **Objective** – criticizing someone else's work can be difficult. Care should be taken that defects are objective, non-judgmental and unemotional. e.g. don't say "**your program crashed**" say "**the program crashed**" and don't use words like "**stupid**" or "**broken**".
- ✓ **Specific** – one report should be logged per defect and only one defect per report.
- ✓ **Concise** – each defect report should be simple and to the point. Defects should be reviewed and edited after being written to reduce unnecessary complexity.
- ✓ **Persuasive** –the pinnacle of good defect reporting is the ability to present them in a way that makes developers want to fix them

CHARACTERISTICS OF A GOOD BUG REPORT

- ✓ **Reproducible** – the single biggest reason for developers rejecting defects is because they can't reproduce them. As a minimum, a defect report must contain enough information to allow anyone to easily reproduce the issue if it can be reproduced at will. Timing issues may be hard to reproduce but in that case there should be other information that can help in determining where in the code the error may be
- ✓ **Explicit** – defect reports should state information clearly or they should refer to a specific source where the information can be found. e.g. "click the button to continue" implies the reader knows which button to click, whereas "click the 'Next' button" explicitly states what they should do.

Sample Bug Report Template

Bug Report Template	
Bug ID	
Module	
Requirements	
Test Case Name	
Release	
Version	
Status	
Reporter	
Date	
Assign To	
CC	
Severity	
priority	
Server	
Platform	
Build No.	
Test Data	
Attachment	
Brief Description	
Navigation Steps	
Observation	
Expected Result	
Actual Result	
Additional Comments	

DEFECT/BUG MANAGEMENT

- ✓ Defects need to be handled in a methodical and systematic fashion
- ✓ There's no point in finding a defect if it's not going to be fixed
- ✓ There's no point getting it fixed if you don't know it has been fixed and there's no point in releasing software if you don't know which defects have been fixed and which remains.

How will you know?

The answer is to have a defect tracking system

The simplest can be a database or a spreadsheet. A better alternative is a dedicated system, which enforce the rules and process of defect handling and makes reporting easier. Some of these systems are costly but there are many freely available variants.

Test Case

The tests are usually defined in a document called Test Plan which spells out all the details of the testing to be done, schedules and milestones, responsibilities, testing systems and installation and setup, and the list of test cases to be executed.

- ✓Test cases can be manual where a tester follows conditions and steps by hand or automated where a test is written as a program to run against the system.
- ✓Tests are usually first defined in a document and often have to be approved by developers and/or testers.
- ✓Automated tests may have all the documentation in the code implementing the test case and in the comments.

Fields of a Test Case

There is no specific Rule for it.

But the common information that all test cases should have:

- ✓ **Unique Test Case name and/or ID:** Each test case needs an identifier. Some tests will have both an id and a meaningful name which is derived from the requirement or the functionality being tested.
- ✓ **Test Scenario and test summary or description:** This description should specify what behavior/functionality will be tested by this test case.
- ✓ **Pre-requisites or setup:** What needs to be setup first, such as previous input or commands to the system to execute this test case.
- ✓ **Test data or inputs:** Data to be used for this test case.
- ✓ **Execution Steps:** The steps that have to be done against the system to test the behavior for this test case.
- ✓ **Expected behavior/Result:** what is expected to happen after execution
- ✓ **Assumptions:** Any assumptions that are made about the system or the test case. For example, data dependency or other test case dependency.
- ✓ **Actual results:** what actually happened when the test case was run.
- ✓ **Status:** What is the current status of the test case such as passed, failed, or not tested.

TEST CASE EXERCISE

Let's say we are to test a login functionality that was designed to look like below:

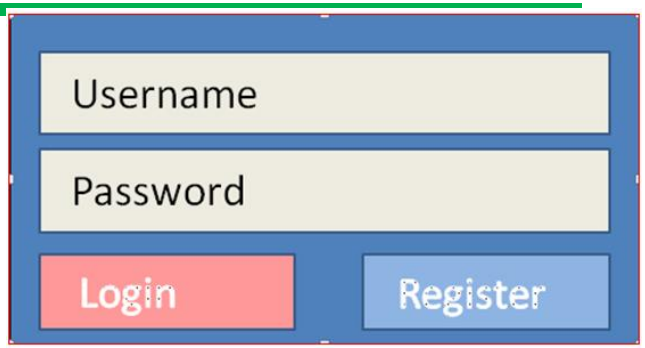
So we have a username and a password fields.
The username is a valid email address and no longer than max valid email address and the password has to be at least 8 characters and no more than 12 characters with only letters and numbers allowed.
Once both registered username and password are entered and Login button clicked, the welcome page will be displayed.

The design indicates that if either username or password is incorrect, we should get the following message above the login area above and the username and password fields should be cleared:

The Username and/or password you entered does not match our records. Try again.

Test Case Exercise

So we define our Test Scenario and Test Cases as follow:



Test Scenario: Verify the login functionality

Test Case 1: Click on Login without typing in username or password

Test Case 2: Type in correct username and nothing for password

Test Case 3: Type in correct password and nothing for username

Test Case 4: Type in incorrect username and incorrect password

Test Case 5: Type in correct username and incorrect password

Test Case 6: Type in incorrect username and correct password

Test Case 7: Type in correct username and correct password

This is not a complete set of possible test cases. For example a username that is not correctly formatted email address; username greater than max size for email address; password less than 8 characters; etc.

Test Case Exercise

Your **Test Case 1** would then be defined something like:

Test Case Name/ID	T1-ClickLogin
Test Scenario	Login:
Test Case description	Missing both username and password
Pre-requisites	None
Execution Steps	1. Click Login button 2. Check the error message 3. Check that there is no text in the username and password fields
Expected behavior/Result	Message will be displayed above login box "The Username and/or password you entered does not match our records. Try again." Both the username and password fields will be cleared
Assumptions	The GUI design will be tested in a separate test scenario
Actual results	
Status	Not Tested

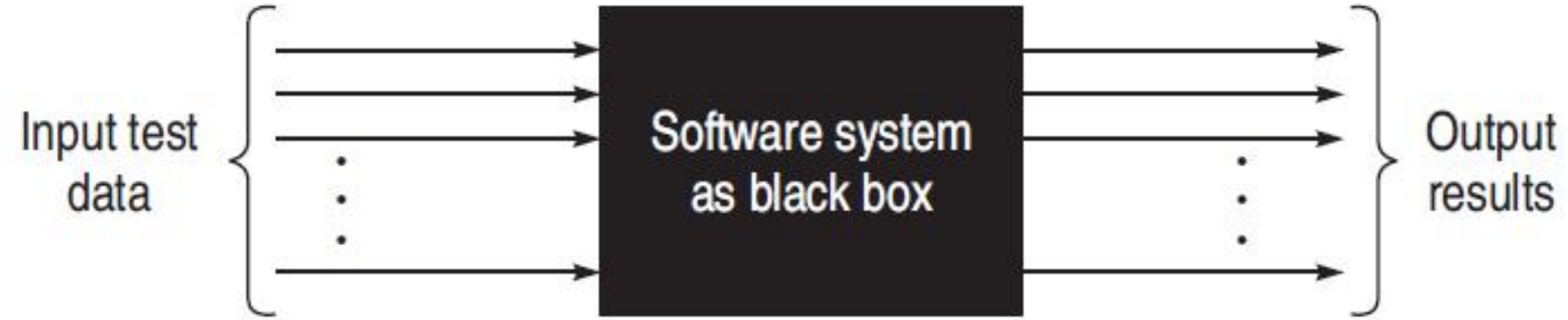
So Now You Try!!!

TESTING CATEGORY & TECHNIQUES

Testing Category	Techniques
Dynamic testing: Black-Box	Boundary value analysis, Equivalence class partitioning, State table-based testing, Decision table-based testing, Cause-effect graphing technique, Error guessing.
Dynamic testing: White-Box	Basis path testing, Graph matrices, Loop testing, Data flow testing, Mutation testing.
Static testing	Inspection, Walkthrough, Reviews.
Validation testing	Unit testing, Integration testing, Function testing, System testing, Acceptance testing, Installation testing.
Regression testing	Selective retest technique, Test prioritization.

BLACK-BOX TESTING

- ✓A “black-box” means that one cannot see the internals of the system or inside the box.
- ✓A test or a testing activity that is considered “black-box”, only checks that **given some input to the SW system, there is expected output.**
- ✓This technique considers only the functional requirements of the software or module.



- ✓An example of such a test is User Acceptance Testing (UAT) where the person doing the testing has no access to the code, is not concerned how the system was implemented, but rather is testing that the system meets the user requirements and works as expected for the typical input that the system needs to work with.

LOOK AT EXAMPLE

Let's assume that we are building a small program that employees at a company can use to determine how much their insurance will cost. Here are the requirements for the application.

Requirement #1 An employee must have health insurance to get vision or dental insurance.

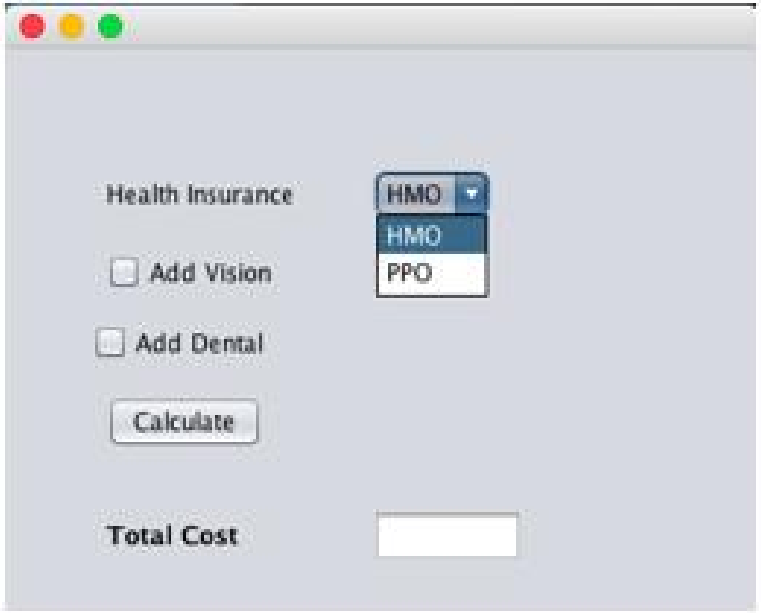
Requirement #2 There are two types of health insurance: HMO costs $100 a month; PPO costs $150 a month.

Requirement #3 Vision costs $25 a month

Requirement #4 Dental costs $20 a month

LOOK AT EXAMPLE

The user interface might look like this. The user selects a health insurance plan and might check boxes to get vision or dental insurance. Then the user clicks the Calculate button. The calculated cost appears in the text field beside the label “Total Cost”.

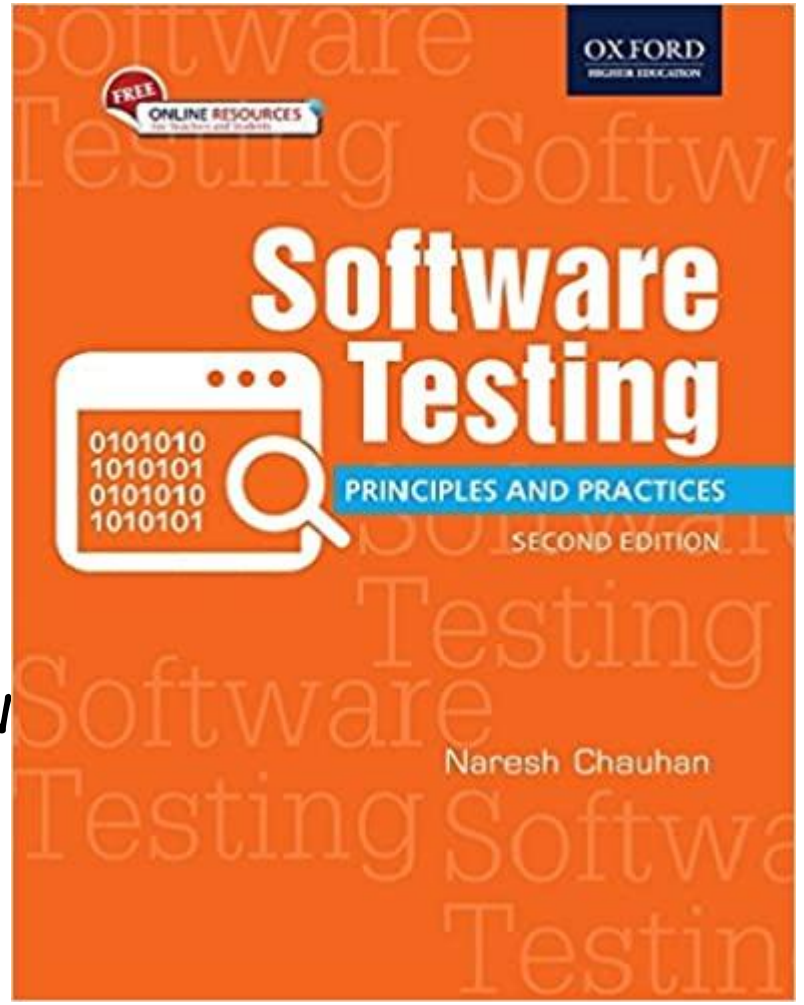


BLACK-BOX TESTING

- ✓ Black-box testing attempts to find errors in the following categories:
  - To test the modules independently
  - To test the functional validity of the software so that incorrect or missing functions can be recognized
  - To look for interface errors
  - To test the system behavior and check its performance
  - To test the maximum load or stress on the system
  - To test the software such that the user/customer accepts the system within defined acceptable limits

Black-Box Testing : Techniques

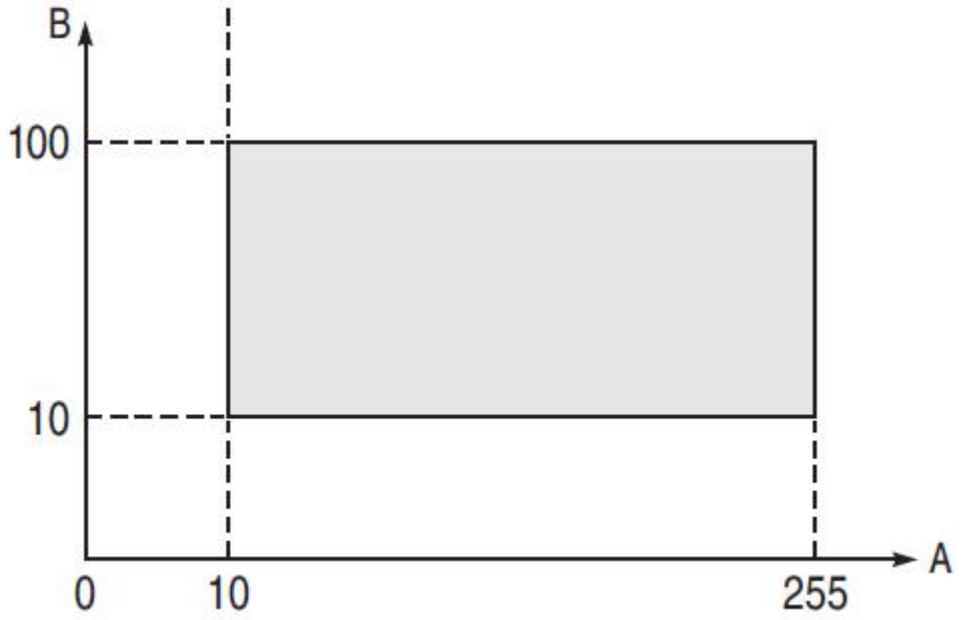
1. Boundary Value Analysis (BVA)
2. Equivalence Class Testing
3. State Table-Based Testing
4. Decision Table-Based Testing
5. Cause-Effect Graphing Based Testin
6. Error Guessing



I will upload the softcopy in google classroom

BOUNDARY VALUE ANALYSIS (BVA)

- ✓ Test cases designed with boundary input values have a high chance to find errors.
- ✓ means that most of the failures crop up due to boundary values.
- ✓ boundary means the maximum or minimum value taken by the input domain.
- ✓ For example, if A is an integer between 10 and 255, then boundary checking can be on 10(9,10,11) and on 255(256,255,254). Similarly, B is another integer variable between 10 and 100, then boundary checking can be on 10(9,10,11) and 100(99,100,101)



BOUNDARY VALUE ANALYSIS (BVA)

BVA offers several methods to design test cases:

- ✓ Boundary Value Checking (BVC)
- ✓ Robustness Testing Method
- ✓ Worst-case Testing Method

BVA: Boundary Value Checking

- ✓ the test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain.
- ✓ The variable at its extreme value can be selected at:

  (a) Minimum value (Min)

  (b) Value just above the minimum value (Min+)

  (c) Maximum value (Max)

  (d) Value just below the maximum value (Max−).

BVA: Boundary Value Checking

Let us take the example of two variables, A and B. If we consider all the above combinations with nominal values, then following test cases can be designed:

1. A_{nom}, B_{min}
2. A_{nom}, B_{min+}
3. A_{nom}, B_{max}
4. A_{nom}, B_{max-}
5. A_{min}, B_{nom}
6. A_{min+}, B_{nom}
7. A_{max}, B_{nom}
8. A_{max-}, B_{nom}
9. A_{nom}, B_{nom}

It can be generalized that for ***n*** variables in a module, ***4n* + 1** test cases can be designed with boundary value checking method.

BVA: Robustness Testing Method

- ✓ Is an extension of BVC such that boundary values are exceeded as:
- ✓ The variable at its extreme value can be selected at:

  (a) A value just greater than the Maximum value **(Max+)**

  (b) A value just less than Minimum value **(Min−)**

So if we consider the previous example then we can add following cases :

10. A_{max+}, B_{nom}
11. A_{min-}, B_{nom}
12. A_{nom}, B_{max+}
13. A_{nom}, B_{min-}

It can be generalized that for ***n*** variables in a module, **6*n* + 1** test cases can be designed with Robustness Testing method.

BVA: Worst-Case Testing Method

- ✓ We can again extend the concept of BVC by assuming more than one variable on the boundary. It is called worst-case testing method.

So if we consider the previous example then we can add following cases to the list of 9 test cases designed in BVC as:

10. A_{min}, B_{min}
11. A_{min+}, B_{min}
12. A_{min}, B_{min+}
13. A_{min+}, B_{min+}
14. A_{max}, B_{min}
15. A_{max-}, B_{min}
16. A_{max}, B_{min+}
17. A_{max-}, B_{min+}
18. A_{min}, B_{max}
19. A_{min+}, B_{max}
20. A_{min}, B_{max-}
21. A_{min+}, B_{max-}
22. A_{max}, B_{max}
23. A_{max-}, B_{max}
24. A_{max}, B_{max-}
25. A_{max-}, B_{max-}

It can be generalized that for ***n*** variables in a module, 5^n test cases can be designed with Worst-Case Testing method.

EXAMPLE

A program reads an integer number within the range [1,100] and determines whether it is a prime number or not. Design test cases for this program using **BVC**, **robust testing**, and **worst-case testing** methods.

Solution:

(a)Test cases using BVC

Total # of test case will be 4n+1 = 5 [as n=1 here]

Min value = 1
Min^{+} value = 2
Max value = 100
Max^{-} value = 99
Nominal value = 50–55

min and max values

Test Case ID	Integer Variable	Expected Output
1	1	Not a prime number
2	2	Prime number
3	100	Not a prime number
4	99	Not a prime number
5	53	Prime number

Test cases

EXAMPLE

(b) Test cases using Robust testing

Total # of test case will be 6n+1 = 7 [as n=1 here]

Min value = 1
Min^- value = 0
Min^+ value = 2
Max value = 100
Max^- value = 99
Max^+ value = 101
Nominal value = 50–55

min and max values

Test Case ID	Integer Variable	Expected Output
1	0	Invalid input
2	1	Not a prime number
3	2	Prime number
4	100	Not a prime number
5	99	Not a prime number
6	101	Invalid input
7	53	Prime number

Test cases

EXAMPLE

(c) Test cases using worst-case testing

Total # of test case will be $5^n = 5$ [as n=1 here]

So, the number of test cases will be same as BVC.

EXAMPLE

A program computes a^b where a lies in the range [1,10] and b within [1,5].Design test cases for this program using **BVC**, **robust testing**, and **worst-case testing** methods.

Solution:

(a)Test cases using BVC : Total # of test case will be 4n+1 = 9 [as n=2]

	a	b
Min value	1	1
Min^+ value	2	2
Max value	10	5
Max^- value	9	4
Nominal value	5	3

min and max values

nominal value and all the cases

Test Case ID	a	b	Expected Output
1	1	3	1
2	2	3	8
3	10	3	1000
4	9	3	729
5	5	1	5
6	5	2	25
7	5	4	625
8	5	5	3125
9	5	3	125

Test cases

EXAMPLE

(b) Test cases using Robust testing: Total # of test case will be 6n+1 = 13 [as n=2]

	a	b
Min value	1	1
Min^- value	0	0
Min^+ value	2	2
Max value	10	5
Max^+ value	11	6
Max^- value	9	4
Nominal value	5	3

min and max values

Test Case ID	a	b	Expected output
1	0	3	Invalid input
2	1	3	1
3	2	3	8
4	10	3	1000
5	11	3	Invalid input
6	9	3	729
7	5	0	Invalid input
8	5	1	5
9	5	2	25
10	5	4	625
11	5	5	3125
12	5	6	Invalid input
13	5	3	125

Test cases

EXAMPLE

(c) Test cases using worst-case testing

Total # of test case will be $5^n = 25$ [as n=2]

Test Case ID	a	b	Expected Output
19	9	4	6561
20	9	5	59049
21	10	1	10
22	10	2	100
23	10	3	1000
24	10	4	10000
25	10	5	100000
17	9	2	81
18	9	3	729

	a	b
Min value	1	1
Min^+ value	2	2
Max value	10	5
Max^- value	9	4
Nominal value	5	3

min and max values

DECISION TABLE-BASED TESTING

- ✓ Decision table is another useful method to represent the information in a tabular method.
- ✓ It has the specialty to consider complex combinations of input conditions and resulting actions..

Formation of decision Table:

		Rule 1	Rule 2	Rule 3	Rule 4	...
Condition Stub	C1	True	True	False	I	
	C2	False	True	False	True	
	C3	True	True	True	I	
Action Stub	A1		X			
	A2	X			X	
	A3			X		

Condition stub It is a list of input conditions for which the complex combination is made.
Action stub It is a list of resulting actions which will be performed if a combination of input condition is satisfied.
Condition entry It is a specific entry in the table corresponding to input conditions mentioned in the condition stub. When we enter TRUE or FALSE
Action entry It is the entry in the table for the resulting action to be performed when one rule (which is a combination of input condition) is satisfied. 'X' denotes the action entry in the table.

DECISION TABLE-BASED TESTING

Guidelines to develop a decision table:

- ✓ List all actions that can be associated with a specific procedure (or module).
- ✓ List all conditions (or decision made) during execution of the procedure.
- ✓ Associate specific sets of conditions with specific actions, eliminating impossible combinations of conditions; alternatively, develop every possible permutation of conditions.
- ✓ Define rules by indicating what action occurs for a set of conditions.

DECISION TABLE-BASED TESTING

For designing test cases from a decision table, following interpretations should be done:

- ✓ Interpret condition stubs as the inputs for the test case.
- ✓ Interpret action stubs as the expected output for the test case.
- ✓ Rule, which is the combination of input conditions, becomes the test case itself.
- ✓ If there are k rules over n binary conditions, there are at least k test cases and at the most 2n test cases.

DECISION TABLE-BASED TESTING

Example:

A program calculates the total salary of an employee with the conditions that if the working hours are less than or equal to 48, then give normal salary. The hours over 48 on normal working days are calculated at the rate of 1.25 of the salary. However, on holidays or Sundays, the hours are calculated at the rate of 2.00 times of the salary. Design test cases using decision table testing..

ENTRY

		Rule 1	Rule 2	Rule3
Condition Stub	C1: Working hours > 48	T	F	T
	C2: Holidays or Sundays	T	F	F
Action Stub	A1: Normal salary		X	
	A2: 1.25 of salary			X
	A3: 2.00 of salary	X		

DECISION TABLE-BASED TESTING

The test cases derived from the decision table are given below:

Test Case ID	Working Hour	Day	Expected Result
1	48	Monday	Normal Salary
2	50	Tuesday	1.25 of salary
3	52	Sunday	2.00 of salary

DECISION TABLE-BASED TESTING

Example:

A university is admitting students in a professional course subject to the following conditions:

(a) Marks in Java ≥ 70

(b) Marks in C++ ≥ 60

(c) Marks in OOAD ≥ 60

(d) Total in all three subjects ≥ 220 OR Total in Java and C++ ≥ 150

If the aggregate mark of an eligible candidate is more than 240, he will be eligible for scholarship course, otherwise he will be eligible for normal course.

The program reads the marks in the three subjects and generates the following outputs:

(i) Not eligible

(ii) Eligible for scholarship course

(iii) Eligible for normal course

Design test cases for this program using decision table testing.

DECISION TABLE-BASED TESTING

Solution:

	ENTRY									
	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
C1: marks in Java ≥ 70	T	T	T	T	F	I	I	I	T	T
C2: marks in C++ ≥ 60	T	T	T	T	I	F	I	I	T	T
C3: marks in OOAD ≥ 60	T	T	T	T	I	I	F	I	T	T
C4:Total in three subjects ≥ 220	T	F	T	T	I	I	I	F	T	T
C5:Total in Java & C++ ≥ 150	F	T	F	T	I	I	I	F	T	T
C6:Aggregate marks > 240	F	F	T	T	I	I	I	I	F	T
A1: Eligible for normal course	X	X							X	
A2: Eligible for scholarship course			X	X						X
A3: Not eligible					X	X	X	X		

DECISION TABLE-BASED TESTING

The test cases derived from the decision table are given below:

Test Case ID	Java	C++	OOAD	Aggregate Marks	Expected Output
1	70	75	60	224	Eligible for normal course
2	75	75	70	220	Eligible for normal course
3	75	74	91	242	Eligible for scholarship course
4	76	77	89	242	Eligible for scholarship course
5	68	78	80	226	Not eligible
6	78	45	78	201	Not eligible
7	80	80	50	210	Not eligible
8	70	72	70	212	Not eligible
9	75	75	70	220	Eligible for normal course
10	76	80	85	241	Eligible for scholarship course

Dynamic Testing:
White-Box Testing Techniques

WHITE-BOX TESTING

- ✓A “white-box” or “clear-box” mean that one can see the internals of the system, or in other words, see inside the box.
- ✓So a test or a testing activity that is considered “white-box” is one where the tester has access to the code and uses that knowledge to write tests or conduct the testing activity.
- ✓example of “white-box” test is unit test where developers write test cases, often automated, to verify each unit of code such as a function, method, or class.
- ✓white-box testing ensures that the internal parts of the software are adequately tested.

WHITE-BOX TESTING : NECESSITY

- ✓ white-box testing techniques are used for testing the module for initial stage testing.
- ✓ Since white-box testing is complementary to black-box testing, there are categories of bugs which can be revealed by white-box testing, but not through black-box testing.
- ✓ Errors which have come from the design phase will also be reflected in the code, therefore we must execute white-box test cases for verification of code (unit verification).
- ✓ Some typographical errors are not observed and go undetected and are not covered by black-box testing techniques.
- ✓ White-box testing explores and confirms the testing of logical paths.

White-Box Testing : Logic Coverage Criteria

basic forms of logic coverage:

- ✓ Statement Coverage : is assumed that if all the statements of the module are executed once, every bug will be notified.
- ✓ Decision or Branch Coverage: each branch direction must be traversed at least once.
- ✓ Condition Coverage: Condition coverage states that each condition in a decision takes on all possible outcomes at least once.

STATEMENT COVERAGE

What is Statement Coverage?

is a white box test design technique which involves execution of all the executable statements in the source code at least once.

$$\textbf{Statement Coverage} = \frac{\textit{Number of executed statements}}{\textit{Total number of statements}} \times 100$$

The goal of Statement coverage is to cover all the possible statement in the code.

Unless a statement is executed, we have no way of knowing if an error exists in that statement.

Let's understand this with an example, how to calculate statement coverage.

EXAMPLE STATEMENT COVERAGE

```
scanf ("%d", &x);
scanf ("%d", &y);
while (x != y)
{
      if (x > y)
            x = x - y;
      else
            y = y - x;
}
printf ("x = ", x);
printf ("y = ", y);
```

If we want to cover every statement of this code, then the following test cases must be designed:

Test case 1: x = y = n, where n is any number

Test case 2: x = n, y = m, where n and m are different numbers.

Test case 1 just skips the while loop

Test case 2, the loop is also executed. However, every statement inside the loop is not executed.

So, two more cases need to be designed to cover all statements: designed:

Test case 3: x > y

Test case 4: x < y

Test case 3 and 4 are sufficient to execute all the statements in the code. But, if we execute only test case 3 and 4, then conditions and paths in test case 1 will never be tested and errors will go undetected.

EXAMPLE STATEMENT COVERAGE

```
1 Prints (int a, int b) {
2     int result = a+ b;
3     If (result> 0)
4           Print ("Positive", result)
5     Else
6           Print ("Negative", result)
7     }
```

Take input of two values of a and b

Find the sum of these two values.

If the sum is greater than 0, then print "This is the positive result."

If the sum is less than 0, then print "This is the negative result."

Now, let's see the two different scenarios and calculation of the percentage of Statement Coverage for given source code.

EXAMPLE STATEMENT COVERAGE

Scenario 1:
If a = 3, b = 9

```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

The statements marked in yellow color are those which are executed as per the scenario.
Number of executed statements = 5, Total number of statements = 7
Statement Coverage: 5/7 = 71%

EXAMPLE STATEMENT COVERAGE

Scenario 2:
If a = -3, b = -9

```
1  Prints (int a, int b) {
2  int result = a+ b;
3  If (result> 0)
4      Print ("Positive", result)
5  Else
6      Print ("Negative", result)
7  }
```

The statements marked in yellow color are those which are executed as per the scenario.
Number of executed statements = 6, Total number of statements = 7
Statement Coverage: 6/7 = 85.20%

BRANCH COVERAGE

- ✓ “Branch” in a programming language is like the “IF statements”. An IF statement has two branches: True and False.
- ✓ So in Branch coverage (also called Decision coverage), we validate whether each branch is executed at least once.
- ✓ In case of an “IF statement”, there will be two test conditions:
  - One to validate the true branch and,
  - Other to validate the false branch.
- ✓ Hence, in theory, Branch Coverage is a testing method which is when executed ensures that each and every branch from each decision point is executed.

EXAMPLE BRANCH COVERAGE

```
scanf ("%d", &x);
scanf ("%d", &y);
while (x != y)
{
      if (x > y)
            x = x - y;
      else
            y = y - x;
}
printf ("x = ", x);
printf ("y = ", y);
```

while and *if* statements have two outcomes: True and False. So test cases must be designed such that both outcomes for *while* and *if* statements are tested

Test case 1: x = y
Test case 2: x != y
Test case 3: x < y
Test case 4: x > y

CONDITION COVERAGE

consider the statement: if (x > 0 && y > 0)

We need to confirm that sub-expression in a decision statement is tested for both **true** and **false** outcomes at least once.

We have Boolean expressions:

- x > 0
- y > 0

Each of these should be tested for **true** and **false** values.

Test Case	x	y	x>0	y>0	Condition Result (x > 0 && y > 0)
TC1	1	1	true	true	true
TC2	-1	1	false	true	false
TC3	1	-1	true	false	false
TC4	-1	-1	false	false	false

BASIS PATH TESTING

- ✓ Based on the control structure, a flow graph is prepared and all the possible paths can be covered and executed during testing..
- ✓ is a more general criterion as compared to other coverage criteria and useful for detecting more errors.
- ✓ The objective of basis path testing is to define the number of independent paths, so the number of test cases needed can be defined explicitly to maximize test coverage.
- ✓ Problem: programs that contain loops may have an infinite number of possible paths and it is not practical to test all the paths.

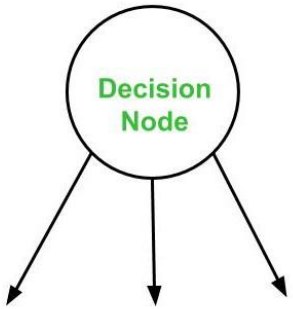
BASIS PATH TESTING

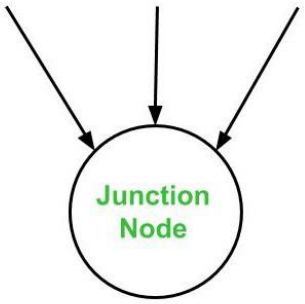
Steps for Basis Path testing

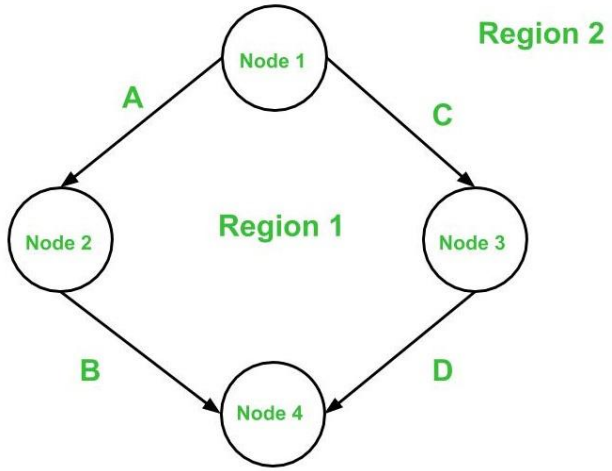
- Draw a **control flow graph** (to determine different program paths)
- Calculate **Cyclomatic complexity** (metrics to determine the number of independent paths)
- Find a basis set of paths (**independent path**)
- Generate test cases to exercise each path

BASIS PATH TESTING : CONTROL FLOW GRAPH

- ✓ A control flow graph (or simply, flow graph) is a directed graph which represents the control structure of a program or module.
- ✓ A control flow graph (V, E) has V number of **nodes/vertices** and E number of **edges** in it. A control graph can also have :
  - **Junction Node** – a node with more than one arrow entering it.
  - **Decision Node** – a node with more then one arrow leaving it.
  - **Region** – area bounded by edges and nodes (area outside the graph is also counted as a region.).







BASIS PATH TESTING : CYCLOMATIC COMPLEXITY

- ✓ The cyclomatic complexity V(G) is said to be a measure of the logical complexity of a program. It can be calculated using three different formulae :

Formula based on edges and nodes

V(G) = e - n + 2*P

Where,
e is number of edges,
n is number of vertices,
P is number of connected components.

Formula based on Decision Nodes

V(G) = d + P

where,
d is number of decision nodes,
P is number of connected nodes.

Formula based on Regions

V(G) = # of regions (R) in the graph

Calculating the number of decision nodes for Switch-Case/Multiple If-Else

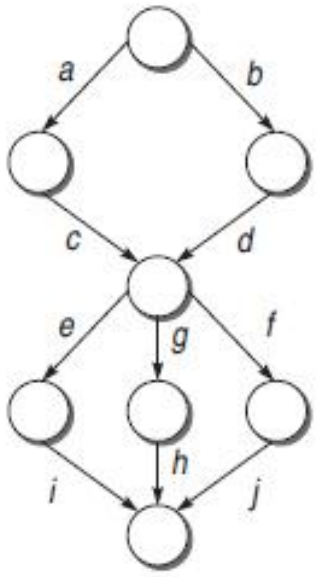
- ✓ If a decision node has exactly two arrows leaving it, then it is counted as one decision node. However, if there are more than 2 arrows leaving a decision node, it is computed using this formula :

d = k – 1, where k is the number of arrows leaving the node.

BASIS PATH TESTING : INDEPENDENT PATH

- ✓ An independent path in the control flow graph is the one which introduces at least one new edge that has not been traversed before the path is defined

- ✓ cyclomatic complexity gives the number of independent paths present in a flow graph.



In the graph, there are six possible paths: *acei, acgh, acfj, bdei, bdgh, bdfj.*
In this case, of the **six** possible paths, only **four** are independent, as the other two are always a linear combination of the other four paths.

If we calculate the cyclometric complexity, it will be same.

BASIS PATH TESTING : EXAMPLE

```
     main()
     {
          int number;
          int fact();
1.        clrscr();
2.        printf("Enter the number whose factorial is to be found out");

3.        scanf("%d", &number);
4.        if(number <0)
5.           printf("Facorial cannot be defined for this number);
6.        else
7.           printf("Factorial is %d", fact(number));
8.   }
```

```
     int fact(int number)
     {
          int index;
1.        int product =1;
2.        for(index=1; index<=number; index++)
3.           product = product * index;
4.        return(product);
5.   }
```

A program for calculating the factorial of a number. It consists of main() program and the module fact().

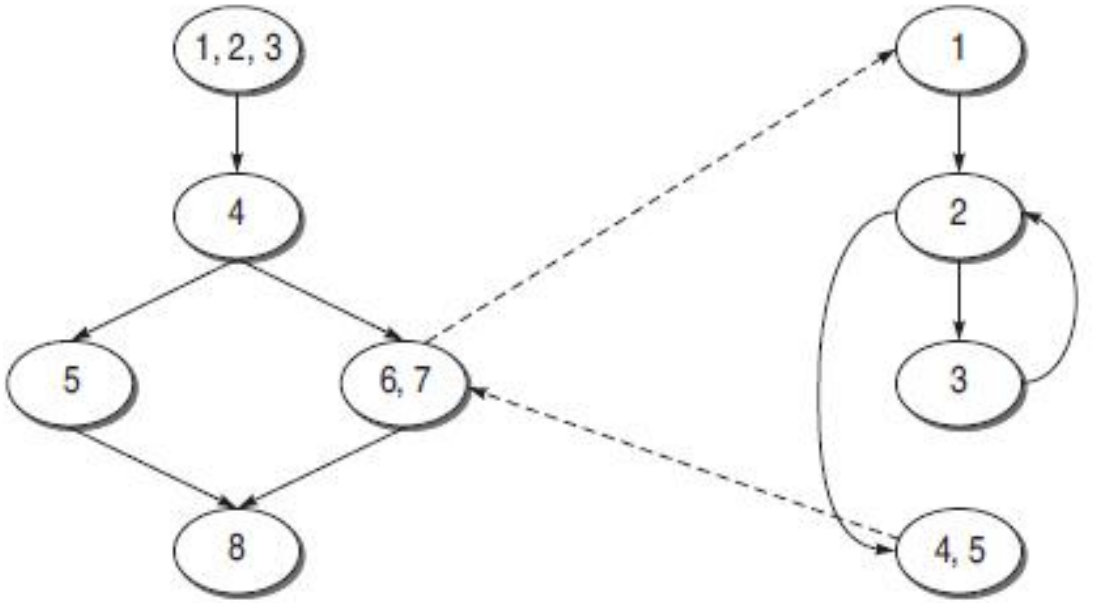
How to Calculate the Cyclomatic complexity?

If a program P consist of multiple components X, Y, and Z. Then we prepare the flow graph for P and for components, X, Y, and Z. The complexity number of the whole program is

$V(G) = V(P) + V(X) + V(Y) + V(Z)$

BASIS PATH TESTING : EXAMPLE

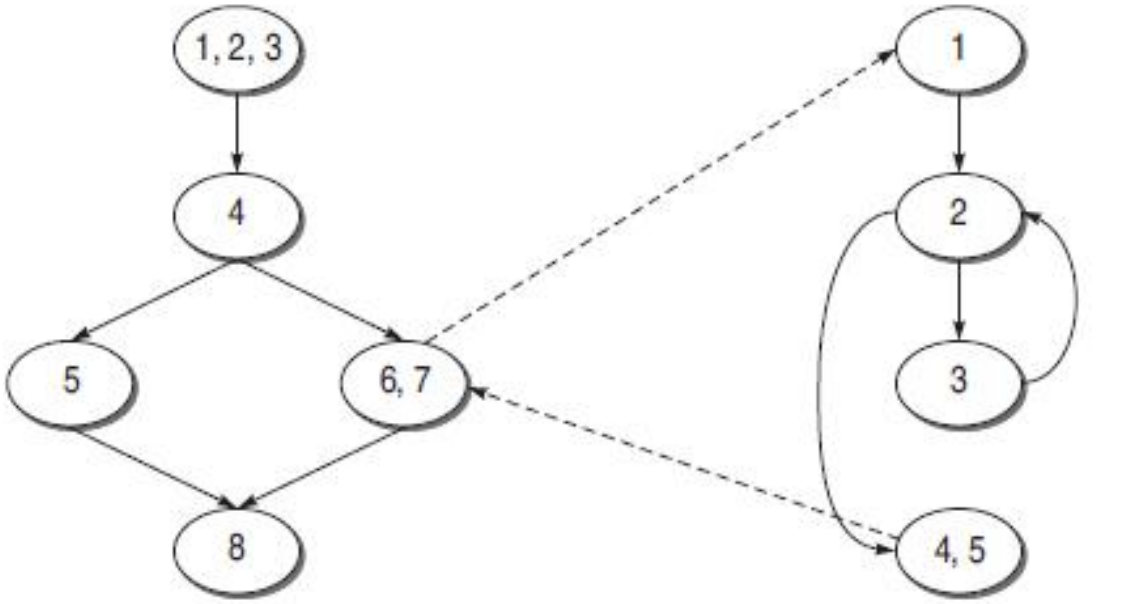
```
     main()
     {
          int number;
          int fact();
1.        clrscr();
2.        printf("Enter the number whose factorial is to be found out");

3.        scanf("%d", &number);
4.        if(number <0)
5.           printf("Facorial cannot be defined for this number);
6.        else
7.           printf("Factorial is %d", fact(number));
8.   }

     int fact(int number)
     {
          int index;
1.        int product =1;
2.        for(index=1; index<=number; index++)
3.           product = product * index;
4.        return(product);
5.   }
```



(a) Flow graph for main ()

(b) Flow graph for fact ()

BASIS PATH TESTING : EXAMPLE



(a) Flow graph for main ()

(b) Flow graph for fact ()

Cyclomatic complexity of main()

(a) $V(M) = e - n + 2p$
$= 5 - 5 + 2$
$= 2$

(b) $V(M) = d + p$
$= 1 + 1$
$= 2$

(c) $V(M)$ = Number of regions = 2

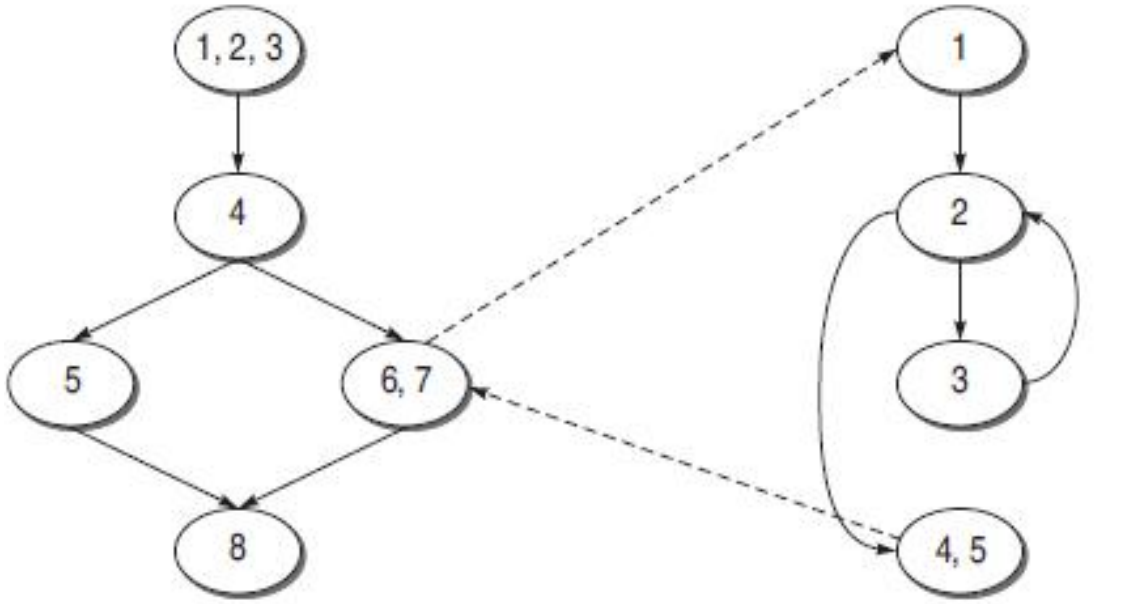
Cyclomatic complexity of fact()

(a) $V(R) = e - n + 2p$
$= 4 - 4 + 2$
$= 2$

(b) $V(R)$ = Number of predicate nodes + 1
$= 1 + 1$
$= 2$

(c) $V(R)$ = Number of regions = 2

BASIS PATH TESTING : EXAMPLE



(a) Flow graph for main ()

(b) Flow graph for fact ()

Cyclomatic complexity of the whole graph considering the full program

(a) V (G) = e – n + 2p
= 9 – 9 + 2 * 2
= 4
= V (M) + V (R)

(b) V (G) = d + p
= 2 + 2
= 4
= V (M) + V (R)

(c) V (G) = Number of regions
= 4
= V (M) + V (R)

EXERCISE

Pathao parcel service announces overnight delivery anywhere inside Dhaka and two days delivery scheme for outside Dhaka (within Bangladesh). The delivery fee is 2 Tk per gram for letters in Dhaka (5 Tk outside of Dhaka), and 10 Tk per gram for parcels in Dhaka (15 Tk outside of Dhaka). The maximum weight they deliver is 200 grams for a letter and 2000 grams for a parcel.

Draw the control flow graph for the above and find the number of independent paths.